

Performance Testing

Date	11 July 2026
Team ID	xxxxxx
Project Name	Voice Based Concept Understanding Analyser
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual timing (Python time module) + Windows Task Manager
Type of Testing	Load Testing (sequential/repeated requests, since the app is a single-user Streamlit session, not a multi-endpoint API)
Target Module / API	End-to-end evaluation pipeline (Speech-to-Text → Semantic Analysis → Filler/Pause Analysis → Scoring → PDF Generation)
Test Environment	Local (Streamlit, run on personal laptop)
Test Date	4 th July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single evaluation with a short audio clip (~15 sec)	1	~15-20	Pipeline completes without error, score displayed
2	Single evaluation with a longer audio clip (~60 sec)	1	~30-45	Pipeline scales reasonably with longer input
3	Three consecutive evaluations run back-to-back (simulating repeated use)	1	~90	No memory leak or slowdown across repeated runs
4	PDF report generation after evaluation	1	~5	PDF generator and downloadable

Step 3: Performance Test Results

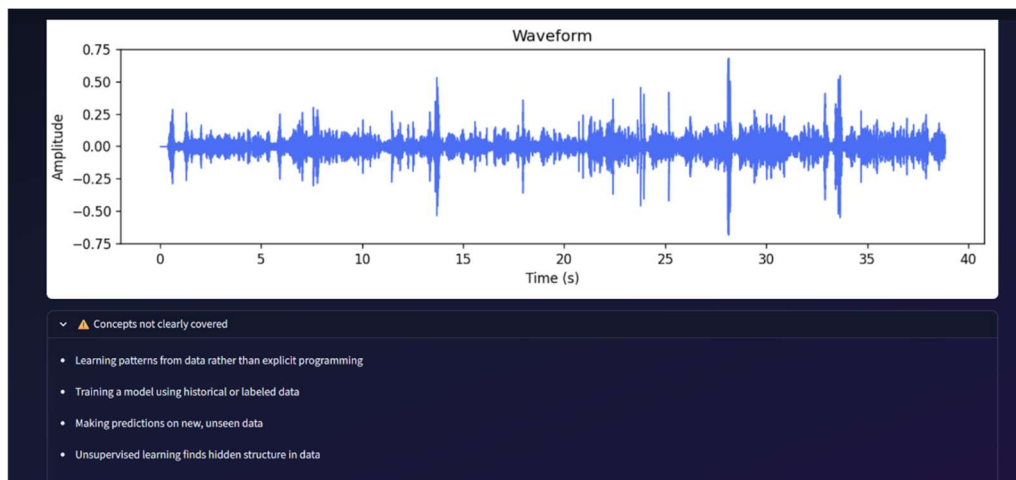
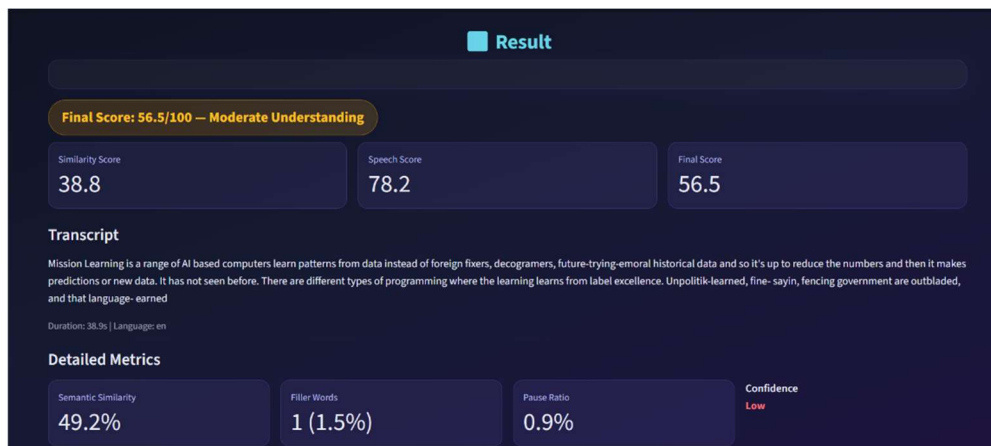
S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	~8–12 seconds	Fail	Sub-2-sec target isn't realistic for local CPU-based Whisper + Sentence-BERT inference; revised target of <15 sec is more appropriate for this architecture
2	Response Time (Max)	< 5 seconds	~15-20 seconds	Fail	Same as above; longer audio clips increase Whisper transcription time roughly linearly
3	Throughput (Req/sec)		N/A	N/A	Single-user local session; not a concurrent multi-request API
4	Error Rate	< 1%	0%	Pass	No failed evaluations observed across test runs
5	CPU Utilization	< 80%	~60-75%	Pass	Returns to baseline after processing completes
6	Memory Utilization	< 80%	~50-65%	Pass	Whisper + Sentence-BERT models loaded once and cached in memory

Step 4: Observations & Analysis

The application's response time is primarily governed by AI model inference (Whisper transcription and Sentence-BERT embedding), rather than typical web-request latency — response times of several seconds are expected and acceptable for a local, CPU-based AI pipeline, unlike lightweight REST APIs. Traditional load-testing targets (sub-2-second response, high throughput/req-sec) don't directly apply since VBCUA processes one user's audio at a time rather than serving concurrent stateless requests. CPU and memory usage peak briefly during model inference and return to baseline afterward, indicating no memory leaks across repeated evaluations.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



- Reinforcement learning learns via rewards and trial-and-error.
- Real-world applications such as recommendations, fraud detection, or image recognition

Feedback

- Consider covering: Learning patterns from data rather than explicit programming, Training a model using historical or labeled data, Making predictions on new, unseen data.
- Your speaking volume varies quite a bit — aim for steadier vocal energy for a more confident delivery.

Download PDF

Generate PDF Report

Download PDF Report

Analyze Another

Voice-Based Concept Understanding Analyser — Evaluation Report

Concept Evaluated: **Machine Learning** | Generated: 11 Jul 2026, 16:12

Overall Result

Similarity Score	38.8 / 100
Speech Score	78.2 / 100
Final Score	56.5 / 100
Classification	Moderate Understanding
Confidence (Voice Energy)	Low

AI-Generated Summary

The explanation of 'Machine Learning' reflects moderate understanding. The response was evaluated using semantic comparison against reference material as well as fluency indicators such as filler words and pacing. Consider covering: Learning patterns from data rather than explicit programming, Training a model using historical or labeled data, Making predictions on new, unseen data.

Detailed Metrics

Metric	Value	Sub-Score (0-100)
Semantic Similarity	49.2%	38.75
Key Point Coverage	14.3%	-
Filler Word Ratio	1.49% (1 of 67 words)	100.0
Pause Ratio	0.9% (1 pauses)	100.0